# Tensor Omics: Team Coding Guidelines

November 2, 2026

## Contents


**1 Compiler: Always Use the Latest gfortran** **3**
- 1.1 Using Docker (Recommended) . . . 3
- 1.2 Native Compilation . . . 3
- 1.3 Fortran Standard Reference . . . 3

**2 File Extensions: `.F90` vs `.f90`** **3**

**3 Code Formatting** **4**
- 3.1 Indentation . . . 4
- 3.2 One Dummy Argument Declaration Per Line . . . 4

**4 Always Use `c_bool` for Logicals** **4**

**5 Always Use Double Precision** **5**

**6 Variable and Argument Naming Conventions** **5**
- 6.1 Prefixes and Suffixes . . . 5
- 6.2 Mapping Arrays . . . 5
- 6.3 Boolean / Selection Arrays . . . 6

**7 Writing for the Code Generator** **6**
- 7.1 The Implementation . . . 6
- 7.2 Documentation Macros (`DM_` prefix) . . . 6
- 7.3 What Comes Out . . . 7
- 7.4 The Workflow . . . 7
- 7.5 When Not to Write a Procedure At All . . . 8
- 7.6 If the Generator Cannot Express It . . . 8

**8 Error Handling** **8**
- 8.1 Error Code Categories . . . 8
- 8.2 `ierr` Comes Last . . . 8
- 8.3 Always Initialise `ierr` First . . . 8
- 8.4 Return Early on Error . . . 8
- 8.5 Reporting the Failing Argument (`arg_pos`) . . . 9
- 8.6 `set_err` vs `set_err_once` . . . 9
- 8.7 Never Silent Errors . . . 9
- 8.8 Avoid GOTO . . . 9

**9 Use of `intent` and `pure`** **10**
- 9.1 Mandatory `intent` . . . 10
- 9.2 Mandatory `pure` for Core Routines . . . 10
- 9.3 Optional Arguments . . . 10

**10 Parallelisation: OpenMP** **10**
10.1 No `do concurrent` . . . 11
10.2 The root . . . 11
10.3 Who writes where . . . 12
10.4 Errors in a root . . . 12
10.5 What a parallel loop must not reach . . . 13
10.6 Hand-written OpenMP directives . . . 13
10.7 Nothing checks that the loop runs in parallel . . . 13
10.8 Callers and their thread count . . . 13

**11 Memory Model: No Copies, No Allocations in Cores** **14**
11.1 No Allocations in Core Routines . . . 14
11.2 No Defensive Copies . . . 14
11.3 Logical Masks . . . 14
11.4 Work Arrays (`tmp_`) . . . 15
11.5 Reusing Output Buffers as Work Arrays (Pointer Reshaping) . . . 15

**12 Macro Usage (`src/macros.h`)** **15**
12.1 Available Macros . . . 16
12.2 Module-Local Constants via `#define` (`CM_` prefix) . . . 16

**13 Derived Types (`type`): Use with Care** **17**

**14 The `f42_utils` Utility Module** **17**

**15 The `f42_safeguard` Module** **17**
15.1 What it guards, and against what . . . 17
15.2 Why the `use` is not optional . . . 18

**16 FORD Documentation** **18**
16.1 Comment Syntax . . . 18
16.2 Module and Subroutine Template . . . 18
16.3 Generating Documentation . . . 19

**17 Testing** **19**
17.1 Where to Test What . . . 19
17.2 Test Naming Convention . . . 19

**18 Merge Request and Collaboration Conventions** **19**

**19 Getting Help: The ask-for-help Channel** **20**

**20 Integrating External Fortran Code from Netlib** **20**
20.1 Shell Archives (shar) . . . 20
20.2 Checking Dependencies . . . 20
20.3 Compiling Legacy Fortran Sources . . . 21
20.4 Linking . . . 21

**21 Further Reading** **21**

**22 Quick-Reference Checklist** **22**

# 1 Compiler: Always Use the Latest gfortran

Always use `gfortran` 15 or newer. Newer compiler versions bring improved standard compliance and better auto-vectorization.

## 1.1 Using Docker (Recommended)

The project provides a pre-built Docker image with gfortran 15 and all required dependencies: no system-level changes are needed. Install Docker as explained for your operating system in the Docker documentation.

**Step 1:** Build the image from the `Dockerfile` in `misc/`:

```
cd misc
docker build -t arch-gfortran -f gfortran.docker .
cd ..
```

**Step 2:** Compile the project inside the container:

```
docker run -it -v $(pwd):/opt -w /opt arch-gfortran ./build.sh
```

## 1.2 Native Compilation

If you already have `gfortran` $\geq$ 15 installed, compile directly with `build.sh`. It compiles all files in `src/`, places compiled objects under `build/<compiler>/`, and creates a symbolic link to the resulting shared library (`libtensor_omics.so`) so that Python and R always find the same path.

**Arch Linux** (rolling release — always has the latest GCC):

```
sudo pacman -S gcc-fortran
```

**macOS** (via Homebrew):

```
brew install gcc
```

On Ubuntu/Debian, gfortran 15 is not available in the standard repositories. Use the Docker path instead.

**Default** — gfortran without optimisation flags:

```
./build.sh
```

**Maximum performance** — gfortran with optimisation flags:

```
./build.sh --max-performance
```

**Intel Fortran Compiler** — with maximum performance flags:

```
./build.sh --max-performance FC=ifx
```

## 1.3 Fortran Standard Reference

When in doubt about language semantics, consult the official Fortran standard draft:

https://j3-fortran.org/doc/year/26/26-007.pdf

# 2 File Extensions: `.F90` vs `.f90`

Fortran source files in this project use *two* extensions with a precise meaning:

- `.F90` (uppercase F): The file is passed through the C preprocessor before compilation. Use this extension whenever the file contains `#include`, `#define`, or any other preprocessor directive (e.g., `#include "macros.h"`).

- `.f90` (lowercase f): Plain Fortran, *no* preprocessor. Use this for files that contain no macros or includes.

**Rule:** If in doubt, use `.F90`. Do not mix files that use macros with the `.f90` extension, as gfortran will silently skip the preprocessor pass and produce cryptic compilation errors.

# 3 Code Formatting

## 3.1 Indentation

Use **4 spaces** per indentation level. Never use tabs. The project formatter is `fprettify`; run it before committing:

```
fprettify -i 4 -l 1000 <path-to-file>
```

The `-l 1000` flag sets a generous line-length limit so that `fprettify` does not introduce spurious line continuations in long argument lists.

## 3.2 One Dummy Argument Declaration Per Line

Each dummy argument (subroutine/function parameter) must be declared on its own line. This is required so that every argument can carry its own FORD documentation comment:

```
! CORRECT
integer(int32), intent(in) :: n_genes
  !! Total number of genes
real(real64), intent(in) :: expression_vectors(n_axes, n_genes)
  !! Gene expression matrix (n_axes x n_genes)

! WRONG: two arguments on one line cannot be individually documented
integer(int32), intent(in) :: n_genes, n_axes
```

For **local variables** this rule does not apply — grouping related locals on one line is acceptable. Local variables do not need FORD comments, but must still follow the naming conventions (meaningful, descriptive names).

# 4 Always Use `c_bool` for Logicals

Every logical that occupies storage — dummy arguments, locals, module variables, arrays — is declared `logical(c_bool)` from `iso_c_binding`, never the default kind.

```
use, intrinsic :: iso_c_binding, only: c_bool

logical(c_bool), intent(in) :: selection_mask(n_genes)
logical(c_bool) :: found
```

**Why.** A default `logical` is four bytes and `c_bool` is one, so a mask over genes costs a quarter as much — and it is the kind C, Python and R already use, so a mask crosses the boundary with no copy at all. Before this, every generated C wrapper declared an automatic array of default `logical` and converted elementwise on the way in and out.

**Two exceptions, both because there is no storage to save.** A `logical, parameter` is a named constant. A `logical function` result is a scalar, and a condition accepts any logical kind, so nothing forces it either way. Neither is worth the noise.

**A literal needs the kind too.** `.true.` is default kind, and argument association does not convert kinds, so passing it to a `logical(c_bool)` dummy is a compile error — write `.true._c_bool`. Assignment is fine and converts silently, which is why `mask = x > 0.0_real64` needs nothing special.

**Characters are not the same case, and the reason is worth knowing.** There is no matching rule to write `character(kind=c_char)` everywhere. `c_bool` is a different kind from the default logical — one byte

against four — so declaring it is what buys the memory and removes the copy. `c_char` *is* the default character kind, so writing it out would change nothing generated and cost an `iso_c_binding` import in every module. An implementation declares an ordinary `character(len=*)`, and `c_char` appears only where a buffer really is a C buffer: the libzip interface and the view helper in `tox_conversions`.

That equality is asserted rather than assumed. The standard defines `c_char` as the kind for C's `char`, or `-1` where there is none; it does not promise the default kind, and a distinct character kind genuinely exists (`selected_char_kind("ISO_10646")`). So `f42_safeguard` carries a fourth compile-time guard beside the three for `c_int`, `c_double` and `c_double_complex`, and the kinds test proves it fires. Writing the plain spelling is safe *because* the guard is there.

What an implementation *does* need to know about characters is that an argument arrives blank-padded to the buffer width rather than trimmed — see `codegen_guide.md`.

# 5 Always Use Double Precision

All real variables and constants must use the ISO Fortran double precision kind `real64` from `iso_fortran_env`. Never use the default `real` kind or literal constants without a kind suffix.

```
use, intrinsic :: iso_fortran_env, only: real64, int32

real(real64) :: x, y
integer(int32) :: n

! Compile-time constants MUST carry the _real64 suffix
x = 3.141592653589793_real64
y = 2.718281828459045_real64
```

Omitting the `_real64` suffix causes the literal to be evaluated as single precision before being promoted, silently losing bits of accuracy.

# 6 Variable and Argument Naming Conventions

Consistent naming makes the code readable without comments. The following conventions are **mandatory**.

## 6.1 Prefixes and Suffixes

| Pattern | Meaning | Example |
|---|---|---|
| `n_` | Size / count of a dimension | `n_genes`, `n_axes`, `n_families` |
| `i_` | Loop / iteration variable | `i_gene`, `i_axis`, `i_vec` |
| `_idx` | Calculated / derived index | `family_idx`, `out_idx` |
| `tmp_` | Work array (not input or output) | `tmp_perm`, `tmp_loess_y` |
| `_perm` | Permutation vector for `<name>` | `sort_perm`, `gene_perm` |
| `_impl` | Core implementation subroutine, and the module holding it | `k_means_assign_cluster_impl` |
| `_c` | C-interoperable wrapper (generated) | `compute_shift_vector_field_c` |
| `_expert` | Full-control wrapper with all work arrays exposed (generated) | `compute_family_scaling_expert` |
| `ierr` | Error code argument (always this name) | `integer(int32), intent(out) :: ierr` |

## 6.2 Mapping Arrays

Use a descriptive `x_to_y` pattern for mapping arrays:

```
integer(int32), intent(in) :: gene_to_fam(n_genes)
! gene_to_fam(i_gene) = family index of gene i_gene

integer(int32) :: family_idx
family_idx = gene_to_fam(i_gene)
```

### 6.3 Boolean / Selection Arrays

Use the `_mask` or `_selection` suffix for logical index arrays. Every logical that occupies storage is declared `logical(c_bool)` — one byte rather than four, and the kind the bindings already use, so a mask crosses to C, Python and R without being copied:

```
logical(c_bool), intent(in) :: vecs_selection_mask(n_vecs)
logical(c_bool), intent(in) :: axes_selection(n_axes)
```

## 7 Writing for the Code Generator

You write **one** procedure per computation: the implementation. The code generator reads it and writes everything around it — the validating and allocating entry points, the `bind(C)` wrapper, the Python and R bindings, their documentation, and the VS Code snippets. None of those may be edited by hand; a change belongs in the implementation or in its annotations. This section is the whole contract. `codegen_guide.md` in the repository root works through it case by case, with an example for each.

### 7.1 The Implementation

- Name it `foo_impl`, in a module whose name ends `_impl`. That module name is what triggers generation, and the generated module is written to `src/generated/` mirroring the implementation's own path.
- It contains the algorithm and nothing else: it **never validates input** and **never allocates** — nor may anything else in an `_impl` module. It may still report a genuine *runtime* error through `ierr` (a division by zero, a failed external fit); validation is not its job.
- Accept *everything* as explicit arguments: inputs, outputs, and any work arrays. Declare it `pure` unless the algorithm genuinely cannot be; an implementation that holds a parallel loop is declared `M_OMP_PURE` instead (Section 10.2). Purity is derived, not declared, for everything above it.
- An argument named `tmp_<name>` is a **work array**: the allocating entry point allocates it and drops it from the signature the caller sees.
- An argument named `<base>_perm` is a **sorting permutation** for `<base>`: the allocating entry point initialises and heapsorts it for you.

### 7.2 Documentation Macros (`DM_` prefix)

Every constraint the generated layers need is written as a `DM_` macro in the argument's own `!!` comment. They are documentation macros: they expand to the English sentence that ends up in the Fortran, C, Python and R documentation, *and* the generator reads them to emit the matching validation. One statement, one place, no way for the check and the prose to disagree.

| Macro | Purpose |
|---|---|
| `DM_MIN(EXPR)` | Lower bound for this argument |
| `DM_MAX(EXPR)` | Upper bound for this argument |
| `DM_SENTINEL(EXPR)` | One extra value accepted outside the bounds |
| `DM_ALLOW_NAN` | Permit NaN (rejected by default for every real argument) |
| `DM_ALLOW_INFINITE` | Permit $\pm\infty$ (rejected by default for every real argument) |
| `DM_DEFAULT(VAL)` | Default for an `optional` argument |
| `DM_RESULT_SIZE_IS(ARG)` | Only the first `ARG` elements of this output are filled |
| `DM_OUTPUT_FROM(...)` | Compute this argument by calling a sizing routine |
| `DM_REQUIRED_IF_MODE(...)` | This optional argument is required in one mode |
| `DM_PROLOGUE(PROC, MOD)` | Run `PROC` before the implementation, and let it short-circuit |

**Finiteness is the default contract**: every real argument is checked for NaN and infinity unless it opts out. A bound may be an expression, including another argument (`DM_MAX(n_array)`) or an exclusive bound (`DM_MIN(above(0.0_real64))`).

```
pure subroutine quantile_normalization_impl(expr, n_genes, n_replicates, &
    normalized_expr, rank_means, tmp_col, tmp_perm)
  integer(int32), intent(in) :: n_genes
    !! Number of genes (rows)
  integer(int32), intent(in) :: n_replicates
    !! Number of replicates per gene
  real(real64), intent(in)  :: expr(n_replicates, n_genes)
    !! Gene expression matrix
    !! DM_ALLOW_NAN
    !! DM_ALLOW_INFINITE
  real(real64), intent(out) :: normalized_expr(n_replicates, n_genes)
    !! Normalized `expr`
  real(real64), intent(in)  :: rank_means(n_replicates)
    !! The mean of each rank across tissues
  real(real64), intent(out) :: tmp_col(n_genes)      ! work array
  integer(int32), intent(out) :: tmp_perm(n_genes)   ! permutation work array
  ! ... pure implementation: no ierr, no validation, no allocation ...
end subroutine
```

## 7.3 What Comes Out

Two Fortran entry points, and the bindings around them:

- `foo` — validates, allocates the work arrays, sorts the permutations, and calls the implementation. **The entry point a caller reaches for first**, and it takes `ierr` last, after the optional arguments.
- `foo_expert` — validates and calls the implementation with what you supply, allocating and preparing nothing. Emitted only where there is something to take over; where there is not, there is a single wrapper and it is called `foo`.
- `foo_c`, the Python `ctypes` module, the R wrapper and its C `.Call` shim, and the snippets — all from the same source, with the documentation rendered per language.

**Your doc comments are the published API.** An implementation's `!>` and `!!` blocks become the Fortran, C, Python and R documentation verbatim, so write them for a reader of those languages rather than as notes to yourself. A `[[link]]` that names nothing is a build error.

## 7.4 The Workflow

When implementing new scientific functionality, complete these steps *in order* and include all of them in the same Merge Request:

1. **Implementation.** Write the annotated `foo_impl` as above. This is the only layer you write.
2. **Generate.** Run `python helper/generate_code.py`; every `./build.sh` does it for you unless `--skip-code-generatio` is passed. Commit the regenerated files with your change. If a published procedure was renamed, also run `--target snippets`, which is not one of the default targets.
3. **Tests.** Fortran unit tests for numerical correctness, plus Python and R tests — see the Testing section. These stay hand-written.

### 7.5 When Not to Write a Procedure At All

Only add functionality that is genuinely absent in Python and R. Do not wrap:

- Sorting routines (Python `numpy.sort`, R `order()` already exist).
- Generic I/O or string utilities.

### 7.6 If the Generator Cannot Express It

The annotation vocabulary is deliberately about language structure, never about domain concepts. If a procedure's contract cannot be expressed in it, that is a sign the *procedure* needs rethinking rather than the generator needing a special case. Raise it rather than working around it.

## 8 Error Handling

All errors are communicated through a single integer output argument `ierr`. The `tox_errors` module defines all error codes and the functions to set and check them.

### 8.1 Error Code Categories

| Range | Category | Example constant |
|---|---|---|
| 0 | Success | `ERR_OK` |
| 1xx | I/O and file errors | `ERR_FILE_OPEN`, `ERR_READ_DATA` |
| 2xx | Input validation | `ERR_INVALID_INPUT`, `ERR_DIM_MISMATCH`, `ERR_NAN_INF` |
| 3xx | Memory | `ERR_ALLOC_FAIL`, `ERR_POINTER_NULL` |
| 5xxx | Fortran runtime | `ERR_UNIT_NOT_CONNECTED` |
| 9xxx | Internal / unknown | `ERR_INTERNAL`, `ERR_UNKNOWN` |

### 8.2 `ierr` Comes Last

`ierr` is the last argument of every procedure that reports an error, after the optional arguments: an `_impl`, a hand-written procedure, and so every generated wrapper, because the generator copies the argument order of the `_impl`. A caller always finds it in the same place, in Fortran and in C. A call that leaves out an optional argument passes it by keyword, `ierr=ierr`, since by position it would bind to the first optional argument left out. The exceptions are the helpers of `tox_errors`, such as `set_err(ierr, code)`, where `ierr` is the value being set or checked, not the helper's own status.

### 8.3 Always Initialise `ierr` First

Call `set_ok(ierr)` at the start of every subroutine that uses `ierr`:

```
use tox_errors, only: set_ok, set_err, set_err_once, is_err, ERR_INVALID_INPUT

call set_ok(ierr)   ! ierr = ERR_OK = 0
```

### 8.4 Return Early on Error

Check `ierr` after every operation that can fail, and return immediately:

```
call validate_dimension_size(n_genes, ierr)
call validate_all_in_range_real(data, n_genes, ierr)
if (is_err(ierr)) return
```

## 8.5 Reporting the Failing Argument (arg_pos)

The generated entry points do all input validation, and number the argument positions for you. On the rare occasion that something has to be validated by hand, use the `tox_errors` validators rather than an ad-hoc check, and pass `arg_pos`:

```
call validate_dimension_size(n_genes, ierr, arg_pos=1_int32)
call validate_in_range_real(span, ierr, min=0.0_real64, max=1.0_real64, arg_pos=6_int32)
call validate_all_in_range_real(data_points, size(data_points, kind=int32), ierr, arg_pos=2_int32)
```

`arg_pos` must match the argument's position in the public (non-`_impl`) subroutine that callers actually invoke, never its position in whichever internal routine happens to run the check — Python and R index their argument-name tuple with it.

When the valid range of an argument is itself defined by `CM_` macros (Section 12.2), pass those macros as the bounds, so the reported range and the validated range can never disagree.

## 8.6 set_err vs set_err_once

- `set_err(ierr, code)`: *Unconditionally* sets `ierr` to `code`, overwriting any error already stored. Use it only when `ierr` is known to be `ERR_OK` (e.g. the first check after `set_ok`), or when you deliberately want the new code to take precedence.
- `set_err_once(ierr, code)`: Sets `ierr` to `code` *only if `ierr` is currently `ERR_OK`*, preserving the first error encountered. This is the safe default when chaining multiple fallible calls, and is exactly what the `validate_*` routines use internally.

## 8.7 Never Silent Errors

Do not ignore `ierr`. If a callee sets an error, the caller must either propagate it or handle it explicitly. Every downstream caller that receives an `ierr` must check it.

## 8.8 Avoid GOTO

**Never use `GOTO`.** It is prohibited in all new code for the following reasons:

- It creates non-local control flow that is impossible to follow, making error paths untraceable.
- It defeats the structured error model built around `ierr` and early returns.
- It prevents the compiler from performing flow analysis, blocking optimizations and `pure` qualification.
- It cannot leave an OpenMP parallel loop (Section 10): a branch out of one is a compile error.

**How to replace `GOTO` correctly:**

| Pattern to replace | Correct Fortran replacement |
|---|---|
| Forward jump to error label | `call set_err(ierr, ...); return` |
| Loop exit (`GOTO after loop`) | `exit` or `cycle` inside a `do` loop |
| Conditional skip block | `if (...) then ... end if` |
| Multi-level exit | Refactor into a subroutine with early `return` |

```
! WRONG
if (n <= 0) goto 99
call compute(n)
99 continue

! CORRECT
if (n <= 0) then
  call set_err(ierr, ERR_INVALID_INPUT)
  return
```

```
end if
call compute(n)
```

# 9 Use of `intent` and `pure`

## 9.1 Mandatory `intent`

Every argument in every subroutine and function must carry an `intent` attribute:

- `intent(in)`: The routine only reads this argument.
- `intent(out)`: The routine completely overwrites this argument.
- `intent(inout)`: The routine both reads and modifies the argument, and the incoming value matters.

Omitting `intent` disables aliasing checks and is forbidden.

## 9.2 Mandatory `pure` for Core Routines

All *helper* and core subroutines must be declared `pure`. A `pure` procedure:

- Has no side effects (no I/O, no global state changes).
- Is eligible for auto-vectorization by the compiler.
- Allows the compiler to schedule calls more aggressively.
- Is safe to call from a parallel loop: with no side effects, it cannot race through global state.

Remove `pure` only when truly necessary (e.g., I/O, or an impure external library). Allocation is not a reason: `allocate(..., stat=)` is legal in a pure procedure. The one exception is the procedure that holds a parallel loop, which is declared `M_OMP_PURE` (Section 10.2).

## 9.3 Optional Arguments

Fortran does not support default values in argument lists. Use the `M_DEFAULT_VAL` macro to assign defaults after a `PRESENT` check:

```
#include "macros.h"
subroutine my_routine(data, n, span, degree)
  real(real64), intent(in), optional :: span
  integer(int32), intent(in), optional :: degree
  real(real64) :: actual_span
  integer(int32) :: actual_degree

  M_DEFAULT_VAL(span, actual_span, 0.7_real64)
  M_DEFAULT_VAL(degree, actual_degree, 2_int32)
```

**Never** use optional arguments in C wrappers; `bind(C)` is incompatible with them.

# 10 Parallelisation: OpenMP

OpenMP is a build option, off by default: `./build.sh --openmp`. The compiler's OpenMP flag defines `_OPENMP`, and that is the one switch the sources key on. There is no project macro such as `USE_OPENMP`, and no build profile carries an OpenMP flag: the comment in `fpm.toml` says why, and the README's build options say what `--openmp` does not combine with. Without it every loop runs serially, and every OpenMP line below is a comment.

Parallelism is written, never left to the compiler, with the macros of `src/macros.h`. Code written on other branches before these rules adopts them when it merges.

## 10.1 No `do concurrent`

Write a plain `do`; CI refuses a `do concurrent` statement in `src/`. gfortran never threads one, while ifx threads every one once OpenMP is on, unasked: with no bound on its team, in places a parallel loop must not be (below), and in loops whose iterations turned out not to be independent. A loop worth threading is the parallel loop of a root.

## 10.2 The root

A *root* is the outermost procedure of a method, and the only one that holds a parallel loop. Nothing it calls opens another: one parallel region per call stack, never nested. Parallelise the outermost loop whose iterations are independent (over families, points, samples), not an inner one. This is a guideline for the author and the reviewer; the generator does not check it.

A root is an implementation (Section 7.1) with five parts:

```
M_OMP_PURE subroutine column_sums_impl(n_rows, n_cols, values, sums, &
                                       n_omp_threads, tmp_col)
    !$ use omp_lib, only: omp_get_thread_num
    integer(int32), intent(in) :: n_rows
        !! rows of the matrix
    integer(int32), intent(in) :: n_cols
        !! columns of the matrix
    real(real64), intent(in) :: values(n_rows, n_cols)
        !! the matrix
    real(real64), intent(out) :: sums(n_cols)
        !! the sum of every column
    integer(int32), intent(in) :: n_omp_threads
    real(real64), intent(out) :: tmp_col(n_rows, n_omp_threads)
        !! one column of scratch per thread
    integer(int32) :: i_col, i_thread

    !M_OMP_PARALLEL(n_cols > 1) schedule(static) shared(n_cols, values, sums, tmp_col) private(i_thread)
    do i_col = 1, n_cols
        i_thread = 1
        !$ i_thread = omp_get_thread_num() + 1
        tmp_col(:, i_thread) = values(:, i_col)  ! reset in this iteration
        sums(i_col) = sum(tmp_col(:, i_thread))  ! one output slot per iteration
    end do
end subroutine column_sums_impl
```

- `M_OMP_PURE` where `pure` would be. An OpenMP directive is not allowed in a pure procedure, so the macro is `pure` without OpenMP and `impure` with it. The generator refuses a root declared plain `pure`, and writes `M_OMP_PURE` onto its wrappers.
- **A required** `n_omp_threads`, `integer(int32), intent(in)`, with no `!!` and no `DM_` macro: the generator writes its documentation and checks that it is at least 1. It caps the team, and it is how the generator recognises a root.
- **Per-thread work arrays with the thread axis last**, `tmp_<name>(..., n_omp_threads)`, in every build. Without OpenMP the allocating `foo` allocates one column of each (through `M_OMP_N_THREADS`, which only the generator writes); a caller of `foo_expert` sizes them.
- **The loop**, under `!M_OMP_PARALLEL(CONDITION)` on the line directly above the `do`, with the `!` in the source and the loop's own clauses after it on the same line. With OpenMP it expands to this, followed by your clauses, and without OpenMP to a comment:

  ```
  !$omp parallel do default(none) num_threads(n_omp_threads) if(n_omp_threads > 1 .and. (CONDITION))
  ```

  `default(none)` makes every variable the loop touches, except its index, need a clause: `shared` for what every iteration reads and for the outputs, `private` for `i_thread` and other per-iteration scalars, `firstprivate` for a private copy that starts from the value before the loop. `CONDITION` says when

the loop has enough work to share: starting and joining a team costs time, so choose its threshold by measuring, and name it as a `CM_` constant (Section 12.2).

**Write the `schedule` with these clauses**, because the right one depends on the loop's items: `schedule(dynamic)` for items of uneven cost, `schedule(static)` for tiny, equal ones. `dynamic` without a chunk fetches work once per iteration, and on tiny iterations that costs more than the work. A missing clause gives the compiler's default, which can only be slower, never wrong, because outputs go by iteration index; so nothing checks that it is there.

- **The thread index**, 1-based: the first two statements of the loop body are `i_thread = 1` and `!$ i_thread = omp_get_thread_num() + 1`, with `!$ use omp_lib, only: omp_get_thread_num` at the top of the procedure. A `!$` line is compiled only with OpenMP, so a procedure needs no `#ifdef`.

`src/macros.h` must define all three macros, with and without `_OPENMP`; the generator stops with a `MissingMacroError` otherwise. `codegen_guide.md` Section 5.17 shows what the generator makes of a root, and Section 8 what it refuses.

## 10.3 Who writes where

The iterations of a root's loop run in an order, and on threads, that change from run to run. The results stay the same only if:

- **Every output has its own place per iteration**, allocated before the loop and addressed by the *iteration* index: `sums(i_col)`, a column of a matrix. No appends: a shared counter such as `n_found = n_found + 1` races.
- **The thread index addresses only the per-thread work arrays**, never an output.
- **A work array is reset inside the iteration**, before it is read. Which iterations a thread ran before this one depends on the schedule, so whatever its column still holds comes from an unknown iteration.
- **No floating-point sum across the iterations**: no floating-point `reduction` clause, no shared accumulator. The order of the additions would depend on the team, and with it the rounding. Write each iteration's part to its own slot, and add the slots after the loop if a total is needed. An integer `reduction`, such as a count, is exact and fine.

Kept to, these give the same output at 1 thread and at several. On ifx that also takes the precise floating-point model, which every ifx build of the profiles carries (`-assume protect_parens -fp-model precise` in `fpm.toml`; `--override-flags` replaces it): without it, ifx `--max-performance --openmp` vectorised a root's serial path, which runs at 1 thread, differently from its parallel body, and 1551 of 3000 column sums differed between 1 and 2–4 threads. With it, none did. Keep the flags.

The speed-up has a trap of its own: short per-thread work columns, up to a few cache lines, false-share. Neighbouring threads write to the same line, which can eat the whole gain (with columns of 5 doubles, a loop ran slower on 4 threads than serially). Measure before migrating a root with short columns.

## 10.4 Errors in a root

No `return`, `exit` or `goto` may leave a parallel loop (each is a compile error), and one `ierr` written by several threads would race. So:

- **A procedure called per item in the loop takes no `ierr`.** Whatever can be validated is validated before the loop: the generated wrapper does that from the `DM_` macros.
- **A condition of one item is information about that item**, and goes into a *status array* indexed like the iteration, one code per item, such as the `status` of `normalize_all_trajectories`.
- **The root's own `ierr` is set only outside the loop**, before or after it.

## 10.5 What a parallel loop must not reach

- **LOESS**, until its rewrite (#200). The netlib code under `external/loess_netlib` keeps state in `save` variables (`save machin,execnt` in `loessf.f`), which threads would race on. Nothing called from a root may reach `tox_loess_impl`, the module through which the library calls it, nor its wrapper `tox_loess`.
- `matmul`, while ifx's `--max-performance` profile carries `-qopt-matmul`. That flag turns `matmul` into a call to Intel's threaded library, which asks for a team of its own inside the parallel region: at the runtimes' default nesting level it gets one thread and only costs time, and if the host allows nesting it oversubscribes the processors.
- **Tasks** (`!$omp task`, `taskloop`). A task can be suspended while another runs on its thread, or resume on a different thread, so the thread index no longer names a column that only it uses.
- **Another parallel region** (Section 10.2).
- **Random numbers**: a loop that draws them stays serial until reproducible random streams exist.

## 10.6 Hand-written OpenMP directives

They are allowed, and the generator does not check them. Prefer the macro: it writes the three clauses a root's directive needs, and a hand-written one has to carry them itself.

- `num_threads(n_omp_threads)`. Without it the team is as large as `OMP_NUM_THREADS` or the processor count, while `foo` allocated `n_omp_threads` columns of every per-thread work array, so the extra threads write past them. With 2 columns and `OMP_NUM_THREADS=8`, `-fcheck=bounds` stops at the first column past 2 that a thread reaches; without the check, memory is silently overwritten.
- `default(none)`, so no variable is shared by accident.
- `if(n_omp_threads > 1 .and. (...))`. A caller who passes 1 must have the loop run on its own thread alone: that is what lets a forked worker process call the procedure (Section 10.8).

It also keeps every other rule of this section: it stands in a root, declared `M_OMP_PURE`, with the thread index as above. The generator refuses `!M_OMP_PARALLEL(` outside a root, but it does not see a hand-written directive, so nothing catches a misplaced one.

## 10.7 Nothing checks that the loop runs in parallel

A root without a recognised `!M_OMP_PARALLEL(` line runs serially in every build, and no diagnostic says so: the line was forgotten, or misspelled (a blank after the `!`, lower case, a typo) and stays a comment. The results are right; only the speed-up is lost. So check it: in an `--openmp` build, call the root above its threshold and see more than one thread work. A test of that needs enough work per iteration: with 64 trivial items under `schedule(dynamic)`, one thread ran them all.

## 10.8 Callers and their thread count

`n_omp_threads` is the caller's choice. It is at least 1 in every build, and the team never has more threads, whatever `OMP_NUM_THREADS` says. Through `foo`, a build without OpenMP uses one work-array column whatever is passed. A caller of `foo_expert` allocates `n_omp_threads` columns itself; how to choose the count with the build query `max_omp_threads()` is on the API page. Pass 1:

- **when you parallelise around the call yourself**: the root would otherwise ask for a team of its own under each of your threads;
- **in a forked worker process** (R's `parallel::mclapply`, Python's `multiprocessing` with the `fork` start method). After the parent has run a parallel region, a forked child that starts one hangs under gfortran's OpenMP runtime. With 1 the `if` clause keeps the loop on the calling thread, and the child runs.

Everything else about calling a root, from Fortran included, is documented once, on the API page (`concise_project_info.md`).

# 11 Memory Model: No Copies, No Allocations in Cores

## 11.1 No Allocations in Core Routines

**Nothing in an `_impl` module may allocate memory**, helpers included. Allocation is performed exclusively in the generated allocating entry point, and the generator enforces this. This rule:

- Keeps core routines `pure` and deterministic.
- Allows callers to reuse pre-allocated work arrays across multiple calls, avoiding repeated `malloc`/`free` cycles over large datasets.

## 11.2 No Defensive Copies

Avoid creating copies of input arrays. For large omics datasets (tens of thousands of genes, hundreds of tissues), an unexpected copy easily exceeds available memory. Design algorithms to work on slices and views of the original data.

## 11.3 Logical Masks

A logical mask is a `logical` array with the same shape as the data it describes: `.true.` marks an element to include, `.false.` marks it to exclude. Passing a mask alongside the full array lets a routine include or exclude elements in place — with a plain `if (mask(...))` check — instead of first allocating a smaller array and copying the kept elements into it. This saves both the allocation and the copy, which matters at big scale.

Prefer adding a mask argument over asking the caller to pre-filter their data into a separate array. In `tox_relative_axis_plane_tools.F90`, `omics_vector_RAP_projection` takes the full `vecs` matrix together with a `vecs_selection_mask`/`axes_selection_mask`, and only writes the selected entries into the (exactly-sized) output — the input matrix itself is never copied or filtered:

```
logical, dimension(n_vecs), intent(in) :: vecs_selection_mask
    !! `.true.` for vectors where projection is to be computed
...
do i_vec = 1, n_vecs
    if (vecs_selection_mask(i_vec)) then
        ...
        if (axes_selection_mask(i_axis)) then
            projections(i_axis_proj, i_vec_proj) = vecs(i_axis, i_vec)
            ...
```

Similarly, in `tox_data_integration_jsd_impl.F90`, an optional `neighbor_mask` lets `build_residual_histograms_impl` skip specific neighbors while iterating the original, unfiltered array:

```
logical, dimension(n_neighbors, n_points), intent(in), optional :: neighbor_mask
    !! Optional mask to exclude specific neighbors (e.g. for family-wise analysis)
...
filter_neighbors = present(neighbor_mask)
...
if (filter_neighbors) then
    if (.not. neighbor_mask(i_neighbor, i_point)) cycle
end if
```

No subset of `neighborhood_residuals` is ever built — the mask only decides which iterations contribute.

When a smaller, compacted array genuinely has to be produced (e.g. to hand a reduced gene list downstream), use `count(mask)` to size the allocation exactly once, rather than over-allocating or growing it as elements are found, as in `tox_data_tools.F90`:

```
n_genes_kept = count(mask)
allocate(temp_gene_ids(n_genes_kept), stat=ios)
...
do i = 1, n_genes_total
    if (mask(i)) then
        temp_gene_ids(j) = gene_ids(i)
        ...
        j = j + 1
    end if
end do
```

Use masks wherever a routine only needs to act on part of its input: they are cheaper than copying, and they keep the calling convention uniform (same array, same shape, one extra `logical` argument) whether or not filtering is requested.

### 11.4 Work Arrays (`tmp_`)

Work arrays must be pre-allocated by the caller and passed as explicit arguments to core routines. They follow the `tmp_` prefix convention.

```
! tmp_perm holds the reusable sort permutation
integer(int32), intent(out) :: tmp_perm(n_genes)
real(real64),   intent(out) :: tmp_loess_y(n_genes)
```

A work array that each thread of a parallel loop needs its own copy of takes the thread axis last, `tmp_<name>(..., n_omp_threads)` (Section 10.2).

### 11.5 Reusing Output Buffers as Work Arrays (Pointer Reshaping)

When a routine has already been handed a large enough output (or other pre-allocated) buffer, reuse it as scratch space via Fortran pointer bounds-remapping instead of allocating a separate temporary. This is a zero-copy *reshape* of contiguous storage: the pointer sees the same memory under new bounds.

```
! Reinterpret a contiguous buffer as a 2-D work view (no copy, no allocation)
real(real64), dimension(:, :), pointer :: work_view
work_view(1:n_genes, 1:n_tissues) => output_buffer
! A column slice can then serve as a work pointer
tmp_col_ptr => work_view(:, 1)
```

**This reshapes; it does not transpose.** Bounds-remapping reinterprets the linear, column-major storage under the new shape: `work_view(i, j)` is element `(j-1)*n_genes + i` of `output_buffer`, *not* `output_buffer(j, i)`. A genuine transpose reorders elements and therefore requires moving data (e.g. `transpose` into a separate buffer); a pointer view can never transpose. The target must be contiguous.

Only reuse a buffer as scratch when its final output value is not needed until after the scratch use has finished — otherwise the scratch writes corrupt the output.

## 12 Macro Usage (`src/macros.h`)

All source files that use macros must start with:

```
#include <src/macros.h>
```

**Never write a bare `implicit none`**: every module uses `M_IMPLICIT_NONE` instead. It expands to `implicit none (type, external)`, the F2018 form, which additionally requires an explicit interface for every external procedure — so calling one that the compiler has never seen becomes a compile error at the call site rather than a link-time surprise or, worse, a silently wrong ABI.

## 12.1 Available Macros

| Macro | Purpose |
|---|---|
| `M_IMPLICIT_NONE` | `implicit none (type, external)`; use it in every module |
| `M_EXPORT_C` | Exports a hand-written procedure; see `codegen_guide.md`, Part II |
| `M_DEFAULT_VAL(OPT,LOC,DEF)` | Assigns `OPT` to `LOC` if present, else assigns `DEF` |
| `M_ALLOCATE(DECL)` | Allocates `DECL`; sets `ERR_ALLOC_FAIL` and returns on failure |
| `M_CHECK_IO_ERR(CODE)` | After I/O: on a non-zero `ierr`, sets `CODE`, closes `unit`, returns |
| `M_NAN` | IEEE quiet NaN (double) |
| `M_NEG_INF` | IEEE negative infinity (double) |
| `M_POS_INF` | IEEE positive infinity (double) |
| `M_GENE_TO_FAM_SENTINEL` | The `gene_to_fam` value of a gene in no family |
| `M_GENE_TO_FAM_DOC(ARG)` | The standard doc sentence for a `gene_to_fam` argument |
| `M_OMP_PURE` | `pure` without OpenMP, `impure` with it; an OpenMP root's purity |
| `!M_OMP_PARALLEL(CONDITION)` | An OpenMP root's parallel loop, with its locality clauses after it |

The `M_CHECK_*` null checks in `macros.h` are not listed: the generator emits them into the C wrappers, and no hand-written code uses them. Nor is `M_OMP_N_THREADS`, which only the generated allocating wrapper of an OpenMP root uses (Section 10.2).

## 12.2 Module-Local Constants via `#define` (`CM_` prefix)

Global, reusable macros live in `macros.h` with the `M_` prefix. A constant that only makes sense within a single module does not belong there — adding it would clutter a file meant for cross-module reuse. Instead, define it locally, immediately after the `#include` line, using the `CM_` prefix ("Custom Macro"), and keep it next to the module's other constants (its `parameter` declarations):

```
#define CM_METHOD_AVERAGE 0
#define CM_METHOD_WEIGHTED 1
#define CM_METHOD_WARD 2

integer(int32), parameter :: METHOD_AVERAGE  = CM_METHOD_AVERAGE
integer(int32), parameter :: METHOD_WEIGHTED = CM_METHOD_WEIGHTED
integer(int32), parameter :: METHOD_WARD     = CM_METHOD_WARD
```

**Keeping FORD documentation in sync.** FORD does not resolve Fortran `parameter` values, but it does run the C preprocessor before generating docs, so a `CM_` macro referenced inside a doc comment renders as-is. This lets a mode/method table stay anchored to the code instead of to a literal that can drift:

```
!! |      Method      |          Value          |
!! |------------------|-------------------------|
!! | Average / UPGMA  |   CM_METHOD_AVERAGE     |
!! | Weighted / WPGMA |   CM_METHOD_WEIGHTED    |
!! |      Ward        |   CM_METHOD_WARD        |
```

**Use the macro in validation too.** When a `CM_` macro defines the valid values or bounds of an argument, pass the macro itself to the corresponding `validate_*` call instead of repeating the literal:

```
call validate_in_range_int(method, ierr, min=CM_METHOD_AVERAGE, max=CM_METHOD_WARD, arg_pos=7_int32
    )
```

This keeps the default/bound, the validation, and the documentation all traceable to one definition — if a mode is added or reordered, only the `#define` block needs to change.

# 13 Derived Types (`type`): Use with Care

Derived types (`type :: ...`) are a powerful Fortran feature but must be used with restraint in this project:

- **Prohibited** in any subroutine or function that has a C wrapper or is otherwise called from Python or R. C, Python `ctypes`, and R's C interface cannot represent Fortran derived types; passing them across the language boundary is undefined behaviour.
- **Permitted** for pure-Fortran data structures where the extra abstraction genuinely improves readability or correctness (e.g., hash maps, JSON serialisation helpers).
- **Avoid by default** in the main TOX scientific routines, which operate exclusively on plain arrays and matrices.

If you need a compound structure that must cross the language boundary, decompose it into flat arrays and pass its components individually.

# 14 The `f42_utils` Utility Module

`src/f42/utils/` contains general-purpose helper functions shared across the project, split by topic into `f42_math_impl`, `f42_sort_impl`, `f42_random_impl`, `f42_vector_impl` and `f42_stats_impl`: `mean`, `std_dev`, `norm`, `clamp`, `sort_array`, `angle_between`, `is_close`, `logx`, `compute_edf`, etc. `f42_utils_impl` re-exports all five, so one `use` reaches everything.

**Rule:** Before implementing a new helper function anywhere else in the codebase, check whether the family already provides it. If you write a function that is genuinely reusable across modules, add it there rather than creating a module-local copy. This prevents duplication and keeps utility logic in one place. Import from the module that *defines* the helper where you know it, and from the parent where you do not:

```
use f42_math_impl, only: norm, is_close, logx, mean, std_dev
use f42_sort_impl, only: sort_array
```

# 15 The `f42_safeguard` Module

**Any module that names a C kind must** `use f42_safeguard` — any `c_bool`, `c_int`, `c_char` or the rest, whether or not the module has a `bind(C)` interface of its own. The code generator enforces this; a module that names one without the `use` is a build error.

```
module tox_thing_impl
    use f42_safeguard
    use, intrinsic :: iso_c_binding, only: c_bool
```

## 15.1 What it guards, and against what

Eight compile-time assertions, of two kinds, because there are two ways a platform can let us down.

**Four equalities** — `c_int == int32`, `c_double == real64`, `c_double_complex == real64`, `c_char ==` the default character kind. This is the dangerous case: the kind exists, so every declaration compiles, and data crossing a wrapper is silently misinterpreted. Nothing else in the build would say a word. These assertions are also what let the framework write `real64` and plain `character(len=*)` rather than converting at the boundary — the equality is the licence for the cheap spelling.

**Four existence checks** — `c_bool`, `c_size_t`, `c_int64_t`, `c_signed_char` are each asserted positive. `iso_c_binding` gives a C kind the value `-1` where the platform has no counterpart, and a negative kind is a compile error at every single declaration naming it. For `c_bool`, which every logical in the framework now uses, that is an error in nearly every file — none of which did anything wrong — instead of one here naming the kind that is missing.

### 15.2 Why the `use` is not optional

The guards fire when *this* module compiles, not when yours does — so the `use` is not what makes them run. It is what makes them run *first*. A dependency is the only way to order a Fortran build, and a module compiled before the safeguard that names a missing kind fails with the compiler's message about that kind rather than the guard's message about which kind is missing.

That is also why `f42_safeguard` depends on nothing but the intrinsic modules, and must stay that way. It is a load-bearing property, not tidiness: it is what puts the module at the front of every build, and what makes each of the eight guards reachable by the kinds test, which forces each mismatch in turn and requires the build to fail.

The rule was learned the hard way. Before logicals moved to `c_bool`, the only modules naming a C kind were the generated wrappers, every one of which had the `use`; the sweep put C kinds into 39 more compilation units and left the dependency behind. Nothing noticed, because on this platform every kind is present and correct.

## 16 FORD Documentation

**FORD** (Fortran Documentation Generator) produces HTML documentation directly from structured comments. All public procedures and modules must be documented.

### 16.1 Comment Syntax

- `!>` — Starts the description of a module, subroutine, or function.
- `!|` — Continues or extends a description (e.g. a second line after `!>`); also used to document an argument when placed **before** the declaration.
- `!!` — Trail comment placed **after** the declaration (on the following line, indented). This is the preferred way to document individual arguments in this codebase.

In summary: `!|` before, `!!` after. Both are valid FORD syntax; the template below uses `!!` consistently.

### 16.2 Module and Subroutine Template

```
!> Module for computing expression centroids of gene families.
!| Contains the core scientific kernel. C and language wrappers are defined
!| outside the module for compatibility.
module tox_gene_centroids
  use, intrinsic :: iso_fortran_env, only: real64, int32
  M_IMPLICIT_NONE
contains

  !> Computes the element-wise mean for a given set of gene expression vectors.
  !| Returns a single centroid vector of length n_axes.
  pure subroutine mean_vector(expression_vectors, n_axes, n_genes, &
      gene_indices, n_selected_genes, centroid, ierr)
    integer(int32), intent(in) :: n_axes
      !! Number of axes (dimensions)
    integer(int32), intent(in) :: n_genes
      !! Total number of genes in the expression matrix
    real(real64), intent(in) :: expression_vectors(n_axes, n_genes)
      !! Gene expression matrix (n_axes x n_genes)
    integer(int32), intent(in) :: n_selected_genes
      !! Number of genes to include in the centroid
    integer(int32), intent(in) :: gene_indices(n_selected_genes)
      !! Column indices of selected genes in expression_vectors
    real(real64), intent(out) :: centroid(n_axes)
```

```
      !! Output centroid vector (n_axes)
    integer(int32), intent(out) :: ierr
      !! Error code: 0 = success
    ! ... implementation ...
  end subroutine mean_vector

end module tox_gene_centroids
```

Note the use of `!!` (two exclamation marks) to place the documentation comment on the line *after* the declaration. This is the FORD convention used throughout the codebase.

### 16.3 Generating Documentation

Documentation is generated automatically by the CI/CD workflow on every merge to the main branch. **You do not need to run FORD manually.** However, if you want to preview the output locally before submitting an MR:

```
ford ford.yml        # run from project root
open doc/index.html  # review in browser
```

## 17 Testing

The project provides a structured test suite framework. See the test directory README for full details on how to add and run tests.

### 17.1 Where to Test What

- **Algorithm correctness**: Fortran unit tests only. The Fortran test suite is the single source of truth for numerical correctness.
- **Python / R wrappers**: Test only that the function can be called, that the return value is of the correct type and shape, and that reasonable input produces a non-error result.

The wrappers themselves are generated, but **their tests are not** — a generated binding is exactly the place where a mistake is uniform and invisible, so every published procedure needs a Python and an R test that actually calls it. Run everything with `./run_all_tests.sh`.

### 17.2 Test Naming Convention

Each test file is named `mod_test_<module_name>.{f90|F90|py|R}`. Each test function is named `test_<description>`.

## 18 Merge Request and Collaboration Conventions

- **Do not merge your own MR.** Always request at least one reviewer and wait for approval before merging.
- **Always assign experienced developers as reviewers.** Add at least Franz or Aaron as a reviewer on every MR. They have deep knowledge of the codebase and can catch issues early.
- **Do not close review threads yourself.** Only the person who opened a thread (the reviewer) should close it after it has been resolved to their satisfaction.
- **Resolve conflicts before requesting review.** Rebase onto the target branch and fix conflicts yourself.

- **Keep MRs focused.** One logical change per MR. Large refactors should be discussed in an issue before starting the MR.
- **Link to the relevant issue** in the MR description.
- **Update FORD documentation** and tests as part of the same MR as the implementation.
- **Check the CI/CD workflow.** After pushing or updating an MR, always verify that the automated workflow (tests, build, documentation generation) passes. If it fails, *fix it yourself* — do not wait to be told. A failing pipeline blocks the merge and means the code is not yet ready for review.
- **Use GitHub, not GitLab.** Since May 2026 the project is developed exclusively on GitHub. Do not open issues, branches, or MRs on the old GitLab repository — those will not be seen or merged.

# 19 Getting Help: The ask-for-help Channel

The ask-for-help channel is the right place for **any question** — not just programming. Whether it is about scientific context, workflow setup, naming decisions, tooling, or general project conventions, there will always be someone available to help. Do not hesitate to ask.

For *technical problems* (especially compilation errors), do not spend more than 30 minutes debugging alone. Post in the channel with:

1. The exact compiler command and full error message (copy-paste, do not paraphrase).
2. The minimal code snippet that triggers the error.
3. What you have already tried.

Many gfortran errors for `bind(C)`, `pure`, or kind-mismatch issues are non-obvious; the team has likely encountered the same problem before.

# 20 Integrating External Fortran Code from Netlib

Netlib is the canonical archive for legacy scientific Fortran (BLAS, LINPACK, LOESS, etc.). This section explains how to extract, compile, and link Netlib packages so they can be added to `external/`.

## 20.1 Shell Archives (shar)

Netlib sometimes distributes packages as **Shell Archives** (`.shar`): self-extracting shell scripts that reconstruct source files when executed. Each line of actual source is prefixed with a hyphen; the script uses `sed` to strip those prefixes during extraction and write the real files. This format dates to the 1980s and was also a safety measure against accidental execution of embedded shell commands.

Do not copy-paste the text manually. Save it to a file and run it:

```
wget https://netlib.org/a/loess
sh loess          # extracts README, loessf.f, lowesf.f, lowescp.f, ...
```

Perform extraction in a **temporary directory outside the project**. Once compilation is confirmed working, move the relevant files into `external/`.

## 20.2 Checking Dependencies

After extraction, read the `README`. Many Netlib packages omit routines they assumed were already present on the target system. LOESS, for example, depends on LINPACK routines (`dqrsl`, `dsvdc`) that are not included in the distributed archive. Missing symbols cause linker errors such as:

```
undefined reference to 'dqrsl'
undefined reference to 'dsvdc'
```

Fetch the missing `.f` files from Netlib and add them to the compilation.

### 20.3 Compiling Legacy Fortran Sources

Old Fortran (fixed-form, F77) requires special flags:

```
gfortran -O2 -std=legacy -ffixed-line-length-none \
         -fallow-argument-mismatch -fPIC -w \
         -c file1.f file2.f file3.f
```

| Flag | Purpose |
|---|---|
| `-O2` | Standard optimisations. |
| `-std=legacy` | Accepts F77 constructs no longer valid in modern standards. |
| `-ffixed-line-length-none` | Removes the 72-character column limit of fixed-form source. |
| `-fallow-argument-mismatch` | Suppresses fatal errors from implicit-interface argument-type mismatches common in old code. |
| `-fPIC` | Position-independent code; required when object files will be linked into a shared library (`.so`). |
| `-w` | Suppresses warnings. Use it only after the initial integration is stable; omit it while diagnosing problems. |
| `-c` | Compile only (produce `.o` files); do not link. |

### 20.4 Linking

To build a **static library** for inclusion in Tensor-Omics:

```
ar rcs libname.a *.o
```

To build a standalone **executable** for testing:

```
gfortran -o program_name file1.o file2.o file3.o
```

If `undefined reference` errors appear at link time, a dependency is missing. Classic culprits in Netlib packages are additional routines from **BLAS**, **LINPACK**, or utility functions such as `d1mach` — fetch and compile those as well.

## 21 Further Reading

This guide states the conventions. Four documents in the repository go deeper, and each is kept in step with the code:

- `codegen_guide.md` (repository root) — the author's guide to the code generator, case by case: ranges, masks, work arrays, permutations, sizing routines, modes and mode splits, prologues, runtime errors, split families. Every case has a real snippet and shows what comes out. **Read this before adding a procedure.**
- `helper/codegen/README.md` — what the generator produces and how to run it, including what it refuses and why.
- `helper/codegen/emit/README.md` — the language layers: every question the Python and R bindings had to answer (array layout, ownership of `intent(inout)`, null-as-absent, missing values, derived arguments, decoding `ierr`) and what each chose. Written so a third language decides rows instead of rediscovering them.
- `helper/codegen/design/` — the design of record, including every rejected alternative.

# 22 Quick-Reference Checklist

Use this checklist when reviewing or submitting any Fortran code contribution:

- ☐ File extension is `.F90` (macros used) or `.f90` (no macros).
- ☐ `#include <src/macros.h>` is present, and the module declares `M_IMPLICIT_NONE` rather than a bare `implicit none`.
- ☐ All `real` variables use `real(real64)`; all constants carry `_real64`.
- ☐ Every argument has `intent(in|out|inout)`.
- ☐ The implementation is named `foo_impl`, in a module ending `_impl`, and is declared `pure` (`M_OMP_PURE` if it holds a parallel loop).
- ☐ No `do concurrent`. A parallel loop stands only in a root, under `!M_OMP_PARALLEL(...)` (or a hand-written directive with the same clauses), with a required `n_omp_threads`, per-thread work arrays with the thread axis last and reset in each iteration, outputs addressed by the iteration index, no `ierr` and no floating-point sum across iterations, and nothing below it that is parallel, LOESS, `matmul`, a task or random (Section 10).
- ☐ No `GOTO` anywhere.
- ☐ The implementation validates nothing and allocates nothing; where it does report a runtime error, `ierr` is initialised with `set_ok(ierr)` and every failure path returns immediately.
- ☐ Every constraint is stated as a `DM_` macro on the argument it constrains, and every work array is named `tmp_`.
- ☐ No unexplained copies of large arrays.
- ☐ No derived types in anything that reaches C, Python or R.
- ☐ All utility functions are in the `f42_utils` family (or re-use existing ones).
- ☐ FORD comments (`!>`, `!|`, `!!`) are present for the module and every public procedure, written as API prose — they are what Python and R users read.
- ☐ The generator was run and its output committed (`./build.sh`, plus `--target snippets` if a published name changed).
- ☐ Fortran tests cover the new logic.
- ☐ Python/R tests verify call-ability and return type.
- ☐ MR does not self-merge and review threads are not self-closed.